

Neural Acceleration — Standalone Draft

First-principles design. The reason to look at a neural operator here is not that the Sinkhorn path is conceptually broken. It is that the cost is wrong for repeated filtering. In §5.3 the OT resampler costs $\mathcal{O}(LN^2)$ per resampling step, with L Sinkhorn iterations and N particles. A single filter run solves hundreds of nearby OT problems. The particle clouds change, but the structure of the problem does not. That is an amortisation opportunity.

I first considered the supervised route: store particle clouds, weights, and the Sinkhorn transport matrix, then train a network to predict the discrete transport. The current code does not take that route. The implementation in `code/neural_operator/src/` follows the Brenier / Monge–Ampère route. It smooths the empirical source and target measures with KDEs, writes the source as a uniform KDE and the target as a weighted KDE, and trains a conditional map $T(x; c)$ so that the log change-of-variables residual is small. I used a direct- T parameterisation rather than an input-convex potential ϕ because the loss needs $\det J_T$. If $T = \nabla\phi$, then the backward pass through the loss sees Hessians of the potential. In the code, `ConditionalConvexPotentialMap.forward_and_jacobian` returns both $T(x; c)$ and an analytic Jacobian, so the determinant is a first-order object for the implemented map.

What the Monge–Ampère equation is here. The generic statement is simple. If a map $T : \mathbb{R}^d \rightarrow \mathbb{R}^d$ pushes a source density p onto a target density q , then mass conservation gives the change-of-variables identity

$$p(x) = q(T(x)) |\det J_T(x)|. \quad (1)$$

For squared Euclidean transport cost, Brenier’s theorem says the optimal map is the gradient of a convex potential: $T = \nabla\phi$ with ϕ convex, under the usual absolute-continuity conditions. Substituting that structure gives the Monge–Ampère form

$$p(x) = q(\nabla\phi(x)) \det \nabla^2\phi(x). \quad (2)$$

That is the PDE route. I did not implement an explicit potential solve. I implemented the direct-map version of the same mass-conservation equation.

In our resampling problem, the code receives particles $x_i \in \mathbb{R}^d$ with normalized importance weights w_i . The KDE code uses the same scalar Gaussian bandwidth h for the uniform and weighted clouds:

$$K_h(u) = (2\pi h^2)^{-d/2} \exp\left(-\frac{\|u\|^2}{2h^2}\right). \quad (3)$$

The source density is the uniform-smoothed cloud,

$$p_h(x) = \frac{1}{N} \sum_{i=1}^N K_h(x - x_i), \quad (4)$$

and the target density is the same cloud with the particle-filter weights,

$$q_h(x) = \sum_{i=1}^N w_i K_h(x - x_i). \quad (5)$$

The resampling-specific equation is therefore

$$p_h(x) = q_h(T(x; c)) |\det J_T(x; c)|. \quad (6)$$

This is exactly the condition the network is trying to satisfy at collocation points sampled from the uniform KDE. In `losses.py`, `log_kde_uniform` computes $\log p_h(x)$ and `log_kde_weighted` computes $\log q_h(T(x; c))$. The residual I would write from source to target is

$$r(x; c) = \log p_h(x) - \log q_h(T(x; c)) - \log |\det J_T(x; c)|, \quad L_{\text{MA}}(c) = \mathbb{E}_{x \sim p_h} [r(x; c)^2]. \quad (7)$$

The code computes the negative of this residual, `log_det_J - (log_q - log_p.T)`, because its local variable `log_q` is the uniform KDE and `log_p.T` is the weighted KDE at $T(x; c)$. Squaring makes the implemented loss identical.

Input and output of the operator. There are two inputs to the trunk forward pass. The first is the point batch x where the map is evaluated, with shape (B, d) during training collocation or (N, d) during resampling. The second is the context vector c with shape $(C,)$, produced once from the weighted particle cloud. The encoder input in the code is not just coordinates plus log-weights. It builds tokens

$$t_i = [x_i, \log w_i, w_i] \in \mathbb{R}^{d+2}, \quad (8)$$

so the token tensor has shape $(N, d+2)$. `WeightedDeepSetsEncoder` applies a learned per-particle MLP to those tokens, pools with $\sum_i w_i h_i$, concatenates weighted mean, upper-triangular covariance, $\log N$, $\log \text{ESS}$, and $\log h$, then projects to c .

The trunk output is also explicit. `ConditionalConvexPotentialMap.forward_and_jacobian(x, c)` returns

$$T(x; c) \in \mathbb{R}^{B \times d}, \quad J_T(x; c) \in \mathbb{R}^{B \times d \times d}. \quad (9)$$

The Jacobian is analytic. It is not obtained by calling `tape.batch_jacobian` inside the loss. The loss evaluates $\log p_h$ at the input x , evaluates $\log q_h$ at $T(x; c)$, takes the determinant of the analytic $J_T(x; c)$, and squares the residual above.

The module structure is the main architectural choice. Each conditional module keeps W_m fixed with respect to context and only lets context shift biases and positive gates. The Jacobian has the form $W_m^\top D_m W_m$, where D_m is diagonal and nonnegative because the activation derivative and scale parameters are passed through positive maps. This is why the Jacobian is symmetric PSD by construction. I chose that structure because the Monge–Ampère loss asks for $\log \det J_T$ at many collocation points, and I did not want convexity or positive determinant to be a soft afterthought. The encoder is weighted DeepSets: tokens $[x_i, \log w_i, w_i]$ are pooled with the weights and joined with weighted summary statistics before producing c .

Ansatz. The bet was threefold. First, a conditional map $T(x; c)$ should amortise across particle clouds if c is produced by a permutation-invariant set encoder. Relabeling particles should not change the context except by the same relabeling, and the tests enforce that. Second, the Monge–Ampère residual should be trainable without a Sinkhorn teacher. That is why `losses.py` uses KDE evaluations and `slogdet` rather than MSE to a stored transport matrix. Third, the map should stay usable in a filter if the local geometry is controlled. The current regularisers are local-linearity, which checks that $T(x + \delta)$ is close to $T(x) + J(x)\delta$ on the KDE scale, and an eigenvalue lower-bound barrier, which penalizes $\lambda_{\min}(J)$ below a threshold.

Inference path. Once trained, the resampler does one deterministic forward pass. During a filter run I have a weighted cloud $\{(x_i, w_i)\}_{i=1}^N$. The shim in `code/src/resampling/neural_operator_resample.py` computes the same resolution-style bandwidth, encodes the whole cloud once to get c , and applies the same map to every particle:

$$\tilde{x}_i = T(x_i; c), \quad \tilde{w}_i = \frac{1}{N}. \quad (10)$$

It also returns the local Jacobians $J_T(x_i; c)$ for covariance transport. There is no Sinkhorn loop at inference and no $N \times N$ transport matrix. That is the amortisation payoff if the map quality problem is solved.

Tests. The construction tests check the pieces that make the ansatz legal. `test_models.py` checks the nonconditional map: forward and Jacobian shapes, analytic-vs-autodiff Jacobian, symmetry, PSD, and residual initialization. `test_conditional_map.py` repeats this with context fixed, then checks context sensitivity and gradients back to c . `test_encoder.py` checks context shape, permutation invariance, variable N , differentiability, and response to distribution changes. `test_kde.py` checks ESS, weighted moments, Silverman bandwidth sanity, KDE normalization, center-vs-far density ordering, uniform-KDE sampling, and differentiability through particles.

The loss-sanity tests check whether the PDE residual is meaningful. `test_loss_sanity.py` has an identity-map / uniform-weights case where the loss should be exactly zero, and a random-map / uniform-weights case where the loss should be nonzero. `test_oracle_residual.py` builds the exact 1D monotone map by CDF matching; in the latest run the oracle loss was 1.19×10^{-9} . That says the residual can be driven to zero by the correct map. If the learned model plateaus, it is not because the loss has an unavoidable KDE floor.

The quality tests are more severe. `test_resampler.py` is only an interface smoke test: after short training, it checks shapes, uniform weights, PSD local Jacobians, no transport matrix, no ancestor indices, and deterministic outputs. `test_resampler_quality.py` trains for 2000 steps and asks whether the map beats identity on a minimal 1D weighted cloud. `test_quality_normalized.py` repeats the same question with per-cloud normalization and fixed bandwidth. `test_eigenvalue_barrier.py` checks that the barrier is zero near identity, fires at the analytic hinge value for a mock small eigenvalue, and has finite gradients. `test_train_smoke.py` and `test_evaluate.py` check that training runs, outputs stay finite, evaluation metrics are finite, the trained model improves MA residual and mean error on random-cloud evaluation data, and logdet failures stay at zero.

What passed and failed. I ran the suite with `../.venv/bin/pytest -q` under `code/neural_operator`. `pytest.ini` only sets `testpaths`, naming rules, `-ra`, and `--strict-markers`; there are no skip or xfail markers. The run collected 37 tests: 35 passed and 2 failed. `test_quality_normalized.py` reduced loss from 10.74 to 0.304, and h was stable at 0.35, but late gradients spiked to 2852.64 and held-out mean error got worse than identity (2.25 versus 1.00). `test_resampler_quality.py` did not reach its final quality assertion because Keras 3 rejected the checkpoint path; `save_weights` now requires a filename ending in `.weights.h5`. Its trace still showed huge initial loss, variable bandwidth, and large gradient spikes. This is not a benchmarked acceleration result. It is an implementation with useful construction tests and failing quality tests.

Open problem: initialization. `1_initialization.md` records the first failure: the residual map was supposed to start near identity, but random positive module gates made the residual large. The code now uses negative `alpha_init_bias` values and a near-one base scale. The remaining issue is input scaling: unnormalized clouds push `tanh` into saturation. The real fix is to make normalization part of the production training and inference contract, not just a local diagnostic test.

Open problem: quality. `2_quality_test.md` is the practical blocker. The infrastructure smoke test passes, but the map-quality tests do not prove that the operator moves particles toward the weighted target. The current workaround is to adjust synthetic training data, bandwidth policy, and normalization experiments. The real fix is still empirical: make the easy one-dimensional quality ladder pass before claiming anything about filters.

Open problem: gradient supervision. `3_gradient_supervision.md` says the MA residual may be too weak by itself. It constrains the determinant and density ratio, not the full Jacobian. The proposed escalation is Gaussian-to-Gaussian pretraining with analytic maps and analytic Jacobians, or Sobolev-style Jacobian supervision. I would not add that until the quality and bandwidth failures are isolated, because it is a structural change to the training signal.

Open problem: Jacobian lower bound. `4_jacobian_lower_bound.md` identified the old $J \geq I$ bug: a pure residual PSD map can expand but not compress. The current conditional map has the minimal workaround, a learned positive base scale $\lambda(c)$ so that $J \geq \lambda(c)I$ and λ can fall below one. That fixes the most obvious impossibility, but it is still a global scale. If a cloud needs compression in the tails and expansion in the center, a single $\lambda(c)$ may be too crude.

Status. The integration shim in `code/src/resampling/neural_operator_resample.py` compiles and runs as a map-based resampler: it computes a resolution bandwidth, encodes the cloud, returns mapped particles with uniform weights, and exposes local Jacobians for covariance transport. It does not return an $N \times N$ transport matrix. I have not benchmarked it end-to-end against the Sinkhorn baseline, and the current tests do not justify any acceleration claim. The point of including this section is narrower: the design is clear, the failure modes are documented, and the test infrastructure is in place for a future implementer to pick it up without rediscovering the same traps.